OTIMIZAÇÃO DE ROTAS POR COLÔNIA DE FORMIGAS: DESENVOLVIMENTO DE UMA APLICAÇÃO WEB PARA O PROBLEMA DO CAIXEIRO-VIAJANTE E COMPARAÇÃO COM OUTRAS HEURÍSTICAS

ROUTE OPTIMIZATION WITH ANT COLONY OPTIMIZATION: DEVELOPMENT OF A WEB APPLICATION FOR THE TRAVELLING SALESMAN PROBLEM AND COMPARISON WITH OTHER HEURISTICS

[NOME COMPLETO DO AUTOR 1][1]
Orcid: [ORCID]

[NOME COMPLETO DO AUTOR 2 – REMOVER SE NÃO HOUVER][2]
Orcid: [ORCID]

[NOME COMPLETO DO PROFESSOR ORIENTADOR][3]
Orcid: [ORCID]

RESUMO

O problema do caixeiro-viajante consiste em determinar a sequência de visitas de menor custo a um conjunto de locais e, por pertencer à classe NP-difícil, demanda métodos aproximados quando o número de pontos cresce. Trata-se de pesquisa aplicada, experimental e quantitativa, cujo objetivo foi desenvolver uma aplicação web, denominada AntRoute, que resolve o caminho aberto do problema sobre distâncias rodoviárias reais por Otimização por Colônia de Formigas, comparando-a com o Vizinho Mais Próximo, o 2-Opt e um Algoritmo Genético. Construída com ASP.NET Core e Angular, a aplicação obtém a matriz de distâncias do OSRM, exibe as rotas em mapa interativo e transmite a convergência em tempo real. Implementou-se ainda um módulo que, para instâncias aleatórias reprodutíveis, calcula o ótimo exato, executa uma grade de 324 configurações de hiperparâmetros e registra 23 características por instância. Em 40 instâncias de 10 a 50 pontos, a colônia obteve a menor distância média em todos os tamanhos, com gap de 0,03% a 3,15%, contra 0,40% a 5,54% do 2-Opt, 8,25% a 12,24% do Vizinho Mais Próximo e 0,64% a 41,46% do Algoritmo Genético. Em outras 330 instâncias de 5 a 15 pontos e 320.760 execuções, o gap médio foi de 1,456%, com 67,6% de acerto do ótimo; o peso

[1]Graduando em [NOME DO CURSO] pelo Centro Universitário FAESA – Vitória/ES. E-mail: [E-MAIL DO AUTOR 1]
[2]Graduando em [NOME DO CURSO] pelo Centro Universitário FAESA – Vitória/ES. E-mail: [E-MAIL DO AUTOR 2]
[3][TITULAÇÃO E ÁREA DE FORMAÇÃO / INSTITUIÇÃO ONDE CURSOU]. Professor(a) do Centro Universitário FAESA – Vitória/ES. E-mail: [E-MAIL DO ORIENTADOR]

heurístico foi o hiperparâmetro mais influente e a configuração padrão ocupou a 139ª posição entre as 324 avaliadas. Conclui-se que a ferramenta cumpre o objetivo proposto e fornece base de dados para a futura recomendação automática de hiperparâmetros por aprendizado de máquina.

Palavras-chave: otimização por colônia de formigas; problema do caixeiro-viajante; metaheurísticas; otimização de rotas; ajuste de hiperparâmetros.

ABSTRACT

The travelling salesman problem consists of finding the lowest-cost sequence of visits to a set of locations and, since it belongs to the NP-hard class, it requires approximate methods as the number of points grows. This is an applied, experimental and quantitative study whose objective was to develop a web application, named AntRoute, that solves the open-path version of the problem over real road distances using Ant Colony Optimization, and to compare it with the Nearest Neighbor heuristic, the 2-Opt local search and a Genetic Algorithm. Built with ASP.NET Core and Angular, the application obtains the distance matrix from OSRM, displays the routes on an interactive map and streams convergence in real time. A module was also implemented which, for reproducible random instances, computes the exact optimum, runs a grid of 324 hyperparameter configurations and records 23 features per instance. Over 40 instances of 10 to 50 points, the colony achieved the shortest mean distance at every size, with gaps from 0.03% to 3.15%, against 0.40% to 5.54% for 2-Opt, 8.25% to 12.24% for Nearest Neighbor and 0.64% to 41.46% for the Genetic Algorithm. Over a further 330 instances of 5 to 15 points and 320,760 runs, the mean gap was 1.456%, with 67.6% reaching the optimum; the heuristic weight was the most influential hyperparameter and the default configuration ranked 139th among the 324 evaluated. It is concluded that the tool meets the proposed objective and provides a dataset for the future automatic recommendation of hyperparameters through machine learning.

Keywords: *ant colony optimization; travelling salesman problem; metaheuristics; route optimization; hyperparameter tuning.*

1 INTRODUÇÃO

A definição da ordem em que um conjunto de locais deve ser visitado é uma decisão recorrente em atividades de entrega, coleta, manutenção em campo, visitas comerciais e transporte de pessoas. Em todas elas, uma sequência mal planejada aumenta a distância percorrida, o tempo de deslocamento, o consumo de combustível e, por consequência, o custo operacional e a emissão de poluentes. Esse tipo de decisão é modelado, na Pesquisa Operacional, pelo Problema do Caixeiro-Viajante (PCV), conhecido na literatura internacional como *Travelling Salesman Problem* (TSP), que busca a rota de menor custo capaz de visitar cada ponto exatamente uma vez (APPLEGATE *et al.*, 2006).

Apesar do enunciado simples, o PCV pertence à classe dos problemas NP-difíceis (GAREY; JOHNSON, 1979): o número de rotas possíveis cresce de forma fatorial com a quantidade de pontos, de modo que a enumeração completa se torna impraticável já para algumas dezenas de locais. Por isso, aplicações reais recorrem a heurísticas e metaheurísticas, que abrem mão da garantia de otimalidade em troca de soluções de boa qualidade em tempo computacional aceitável (BLUM; ROLI, 2003).

Entre as metaheurísticas, a Otimização por Colônia de Formigas, do inglês *Ant Colony Optimization* (ACO), destaca-se por ter sido concebida justamente sobre o PCV (DORIGO; MANIEZZO; COLORNI, 1996). Inspirada no comportamento de formigas reais, que depositam feromônio no caminho e tendem a seguir trilhas mais marcadas, a ACO constrói soluções de forma probabilística e reforça, ao longo das iterações, as arestas que compõem as melhores rotas. O desempenho do método, contudo, depende de hiperparâmetros como a influência do feromônio (α), a influência da distância (β), a taxa de evaporação (ρ), o número de formigas e o número de iterações, e não há uma configuração que seja a melhor para todas as instâncias (WOLPERT; MACREADY, 1997).

Diante desse contexto, este trabalho busca responder às seguintes questões: como disponibilizar a Otimização por Colônia de Formigas em uma ferramenta acessível, capaz de otimizar rotas sobre distâncias rodoviárias reais? Qual o desempenho da ACO, em qualidade de solução e tempo de execução, quando comparada a heurísticas clássicas e a outra metaheurística? E de que forma é possível gerar, de maneira reprodutível, dados que permitam relacionar as características de uma instância à configuração de hiperparâmetros mais adequada?

O objetivo geral consiste em desenvolver uma aplicação web, denominada AntRoute, que resolva o PCV de caminho aberto por meio da ACO sobre distâncias rodoviárias reais, compare seus resultados com os de outros algoritmos e produza uma base de dados experimental para o estudo dos hiperparâmetros da colônia. Para alcançá-lo, foram definidos os seguintes objetivos específicos:

a) implementar a ACO, na variante *Ant System*, e os algoritmos de referência Vizinho Mais Próximo, 2-Opt e Algoritmo Genético, além de métodos exatos para instâncias pequenas;
b) integrar a aplicação a um serviço de roteamento sobre a malha viária do OpenStreetMap para obter distâncias reais entre os pontos;

c) construir uma interface com mapa interativo que permita cadastrar locais, visualizar as rotas e acompanhar em tempo real a convergência da ACO;

d) comparar os algoritmos quanto à distância obtida, à diferença percentual em relação à melhor solução e ao tempo de execução;

e) desenvolver um módulo de geração de dados que associe características de instâncias ao desempenho de diferentes configurações da ACO, tendo o ótimo exato como referência.

A relevância do estudo é dupla. Do ponto de vista prático, oferece uma ferramenta gratuita, executada no navegador e baseada em dados abertos, que pode apoiar pequenas operações logísticas sem acesso a softwares comerciais de roteirização. Do ponto de vista acadêmico, torna observável o funcionamento interno de uma metaheurística, o que favorece seu uso didático, e organiza um conjunto de dados reprodutível sobre a sensibilidade da ACO a seus hiperparâmetros, etapa preliminar para a recomendação automática de configurações por aprendizado de máquina.

O texto está organizado da seguinte forma: a seção 2 apresenta a revisão da literatura sobre o PCV e os algoritmos empregados; a seção 3 descreve a metodologia, a arquitetura e a implementação da aplicação; a seção 4 apresenta e discute os resultados; e a seção 5 reúne as considerações finais.

2 REVISÃO DA LITERATURA

2.1 O PROBLEMA DO CAIXEIRO-VIAJANTE

Formalmente, dado um conjunto de n locais e uma matriz de custos $D = [d_{ij}]$, em que d_{ij} representa a distância de deslocamento do local i para o local j, o PCV clássico procura uma permutação dos locais que forme um ciclo hamiltoniano de custo mínimo, isto é, uma rota que visite cada local uma única vez e retorne à origem (APPLEGATE *et al.*, 2006; LAPORTE, 1992). Quando $d_{ij} = d_{ji}$ para todo par, o problema é dito simétrico; caso contrário, é assimétrico. Distâncias rodoviárias costumam ser assimétricas, em razão de vias de mão única, retornos e restrições de conversão.

Neste trabalho adota-se a variante de caminho aberto (*open path*), na qual o percurso termina no último local visitado, sem retorno à origem. Essa formulação representa situações como a de um entregador que encerra o expediente no último cliente. Sendo π uma permutação dos locais, o custo a minimizar é dado pela Equação 1:

$$L(\pi) = \Sigma_{t=1}^{n-1} d_{\pi(t),\, \pi(t+1)} \tag{1}$$

Para n locais existem $n!$ permutações possíveis, ou $(n-1)!$ quando o ponto de partida é fixado. Com apenas 15 locais, esse número já supera 87 bilhões de sequências, o que ilustra a explosão combinatória que caracteriza o problema (GAREY; JOHNSON, 1979).

2.2 MÉTODOS EXATOS

Os métodos exatos garantem encontrar a solução ótima. O mais direto é a força bruta, que enumera todas as permutações e cuja complexidade $O(n!)$ a restringe a instâncias muito pequenas. Técnicas de ramificação e poda (*branch and bound*) descartam prefixos de rota cujo custo parcial já excede a melhor solução conhecida, reduzindo o esforço na prática, embora não no pior caso.

Held e Karp (1962) propuseram uma formulação por programação dinâmica que reduz a complexidade para $O(n^2 \cdot 2^n)$ em tempo e $O(n \cdot 2^n)$ em memória. Seja $C(S, j)$ o menor custo de um caminho que parte da origem, visita exatamente os locais do subconjunto S e termina em $j \in S$. A recorrência é apresentada na Equação 2:

$$C(S, j) = \min_{i \in S \setminus \{j\}} [\, C(S \setminus \{j\}, i) + d_{ij} \,] \tag{2}$$

Para o caminho aberto, a solução ótima é o menor valor de $C(V, j)$ entre todos os possíveis destinos finais j, sem a adição do arco de retorno. Ainda que exponencial, o método viabiliza a obtenção do ótimo para instâncias de até algumas dezenas de pontos, o que o torna útil como referência para avaliar a qualidade de heurísticas.

2.3 HEURÍSTICAS CONSTRUTIVAS E DE BUSCA LOCAL

A heurística do Vizinho Mais Próximo (*Nearest Neighbor*, NN) constrói a rota de forma gulosa: a partir do local inicial, move-se sempre para o local ainda não visitado mais próximo. É simples e rápida, com complexidade $O(n^2)$, mas tende a deixar arestas longas no final do percurso, quando restam apenas locais dispersos; Rosenkrantz,

Stearns e Lewis (1977) demonstraram que sua razão de aproximação, no pior caso, cresce logaritmicamente com *n*.

A busca local 2-Opt, introduzida por Croes (1958), parte de uma rota inicial e testa a remoção de duas arestas e a reconexão dos dois segmentos resultantes no sentido inverso. Sempre que a troca reduz o custo, ela é aplicada, e o processo se repete até que nenhuma troca produza melhoria, situação denominada ótimo local. Lin e Kernighan (1973) generalizaram a ideia para trocas de *k* arestas, dando origem a algumas das heurísticas mais eficazes para o PCV.

2.4 METAHEURÍSTICAS E ALGORITMOS GENÉTICOS

Metaheurísticas são estratégias de alto nível que orientam heurísticas subordinadas na exploração do espaço de busca, equilibrando a diversificação, que é a exploração de regiões novas, e a intensificação, que é o aprofundamento em regiões promissoras (BLUM; ROLI, 2003; TALBI, 2009). Diferentemente das heurísticas construtivas, elas são capazes de escapar de ótimos locais, ao custo de maior tempo de processamento.

Os Algoritmos Genéticos (AG), propostos por Holland (1975) e popularizados por Goldberg (1989), mantêm uma população de soluções candidatas que evolui por meio de operadores inspirados na seleção natural. Na seleção por torneio, *k* indivíduos são sorteados e o de melhor aptidão é escolhido como genitor. Para problemas de permutação, como o PCV, o cruzamento precisa preservar a validade da rota; o *Order Crossover* (OX), atribuído a Davis (1985), copia um segmento de um genitor e completa os demais locais na ordem em que aparecem no outro. A mutação por troca (*swap*) permuta dois locais da rota com pequena probabilidade, mantendo a diversidade da população.

2.5 OTIMIZAÇÃO POR COLÔNIA DE FORMIGAS

A ACO baseia-se na estigmergia, forma de comunicação indireta em que os indivíduos modificam o ambiente e reagem a essas modificações. Deneubourg *et al.* (1990) observaram que formigas da espécie *Linepithema humile*, ao depositarem feromônio no trajeto entre o ninho e uma fonte de alimento, acabam por convergir para o caminho mais curto: como as trilhas curtas são percorridas mais vezes por unidade de tempo, acumulam feromônio mais rapidamente e atraem um número crescente de formigas.

Dorigo, Maniezzo e Colorni (1996) transpuseram esse mecanismo para o PCV no algoritmo *Ant System* (AS). A cada iteração, *m* formigas artificiais constroem rotas completas. Estando no local *i*, a formiga *k* escolhe o próximo local *j*, entre os ainda não visitados (conjunto N_i^k), com a probabilidade dada pela Equação 3:

$$p_{ij}^{k} = [\tau_{ij}]^{\alpha} \cdot [\eta_{ij}]^{\beta} \;/\; \Sigma_{l \in N} [\tau_{il}]^{\alpha} \cdot [\eta_{il}]^{\beta} \qquad (3)$$

em que τ_{ij} é a quantidade de feromônio na aresta (i, j) e η_{ij} é a informação heurística, usualmente definida como o inverso da distância, conforme a Equação 4. Os expoentes α e β ponderam, respectivamente, a influência da experiência acumulada pela colônia e a da atratividade imediata da aresta: com $\alpha = 0$ o método degenera em uma heurística gulosa aleatorizada; com $\beta = 0$, apenas o feromônio orienta a busca, o que tende a provocar estagnação precoce (DORIGO; STÜTZLE, 2004).

$$\eta_{ij} = 1 / d_{ij} \tag{4}$$

Após a construção das rotas, o feromônio é atualizado em duas etapas. Primeiro, todas as arestas sofrem evaporação a uma taxa $\rho \in (0, 1)$, o que evita o acúmulo ilimitado e permite à colônia “esquecer” escolhas ruins (Equação 5). Em seguida, cada formiga deposita, nas arestas de sua rota, uma quantidade inversamente proporcional ao comprimento L_k da rota construída (Equação 6), em que Q é uma constante:

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} \tag{5}$$

$$\tau_{ij} \leftarrow \tau_{ij} + \sum_{k=1}^{m} \Delta\tau_{ij}^{k} \tag{6}$$

em que $\Delta\tau_{ij}^{k} = Q/L_k$ se a aresta (i, j) pertence à rota da formiga k, e $\Delta\tau_{ij}^{k} = 0$ caso contrário.

Desde o AS foram propostas variantes que intensificam a exploração das melhores soluções, como o *Ant Colony System* (ACS), com regra de transição pseudoaleatória e atualização local de feromônio (DORIGO; GAMBARDELLA, 1997), e o *MAX-MIN Ant System* (MMAS), que restringe o depósito à melhor formiga e limita o feromônio a um intervalo $[\tau_{min}, \tau_{max}]$ para evitar estagnação (STÜTZLE; HOOS, 2000). Este trabalho implementa o AS original, por sua clareza conceitual e por servir de linha de base para essas extensões.

2.6 AJUSTE DE HIPERPARÂMETROS E SELEÇÃO DE ALGORITMOS

O teorema *No Free Lunch* estabelece que, considerando todos os problemas possíveis, nenhum algoritmo de otimização supera os demais em média (WOLPERT; MACREADY, 1997). Na prática, isso implica que a configuração mais adequada de uma metaheurística depende das características da instância. Rice (1976) formalizou essa questão como o problema de seleção de algoritmos, no qual um espaço de características das instâncias é mapeado para o algoritmo, ou a configuração, de melhor desempenho.

Abordagens de configuração automática, como os algoritmos de corrida (*racing*) de Birattari *et al.* (2002) e a otimização sequencial baseada em modelos de Hutter, Hoos e Leyton-Brown (2011), procuram a melhor configuração para um conjunto de instâncias, mas exigem novas execuções a cada problema. O meta-aprendizado propõe aprender, a partir de experimentos anteriores, a relação entre características da instância e desempenho, permitindo recomendar uma configuração para uma instância nova sem a necessidade de busca (SMITH-MILES, 2008). Para isso, é indispensável uma base de dados que relacione características descritivas, como tamanho, dispersão espacial e estatísticas da matriz de custos, ao resultado de cada configuração, sendo essa a lacuna que o módulo de geração de dados deste trabalho procura preencher.

2.7 ROTEAMENTO SOBRE MALHA VIÁRIA REAL

Grande parte dos estudos sobre o PCV utiliza distâncias euclidianas, o que simplifica a análise, mas se afasta da realidade urbana. Para obter distâncias rodoviárias, é necessário calcular o caminho mínimo na malha viária entre cada par de pontos. O *Open Source Routing Machine* (OSRM) é um motor de roteamento de código aberto

que opera sobre os dados do OpenStreetMap e emprega técnicas de pré-processamento, como as *contraction hierarchies*, para responder a consultas de caminho mínimo em milissegundos (LUXEN; VETTER, 2011). Seu serviço *table* retorna, em uma única requisição, a matriz completa de distâncias e tempos entre um conjunto de coordenadas, insumo direto para os algoritmos de otimização.

Quando se deseja gerar grande volume de instâncias sem depender de um serviço externo, pode-se recorrer à fórmula de Haversine, que calcula a distância ortodrômica entre dois pontos da superfície terrestre a partir de suas latitudes e longitudes (SINNOTT, 1984). Embora subestime a distância rodoviária, ela preserva a estrutura geográfica do problema e permite experimentos reprodutíveis e independentes da rede.

3 METODOLOGIA

3.1 CARACTERIZAÇÃO DA PESQUISA

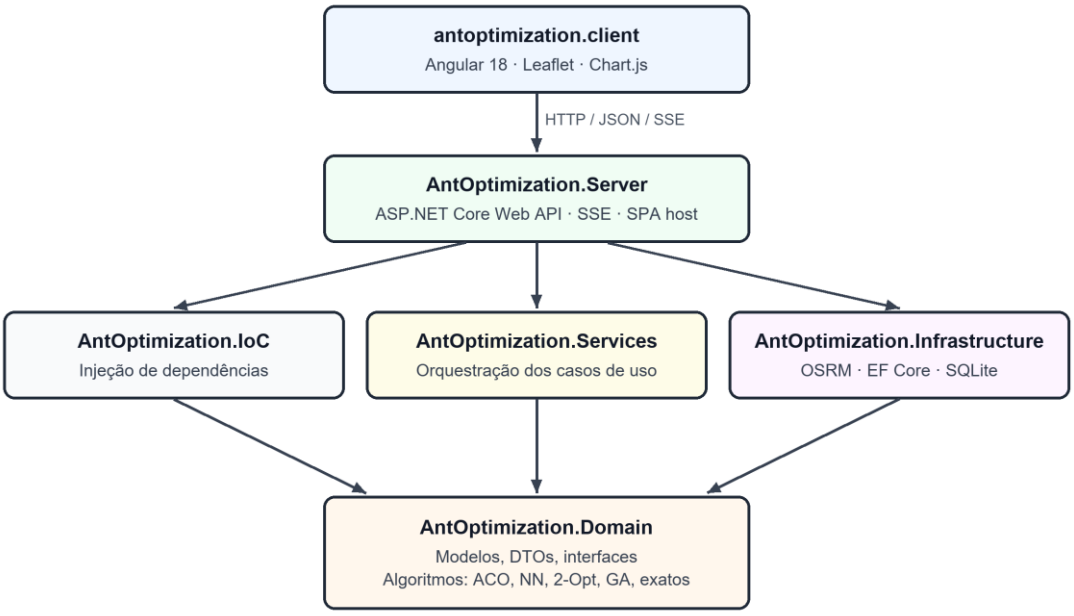
Quanto à natureza, trata-se de uma pesquisa aplicada, voltada ao desenvolvimento de uma solução para um problema prático. Quanto aos procedimentos, é experimental, pois manipula variáveis controladas, como o algoritmo, os hiperparâmetros e o tamanho das instâncias, e observa seus efeitos sobre a qualidade da solução e o tempo de execução. Quanto à abordagem, é quantitativa. O desenvolvimento seguiu as seguintes etapas:

a) levantamento bibliográfico sobre o PCV, métodos exatos, heurísticas e metaheurísticas;
b) definição da arquitetura e das tecnologias da aplicação;
c) implementação dos algoritmos, da integração com o serviço de roteamento e da interface;
d) implementação do módulo de geração de dados experimentais;
e) verificação por testes automatizados e implantação em nuvem;
f) execução dos experimentos, análise e discussão dos resultados.

3.2 ARQUITETURA DA APLICAÇÃO

A aplicação foi organizada segundo os princípios da Arquitetura Limpa (*Clean Architecture*), que propõe a separação do sistema em camadas concêntricas, com as regras de negócio no centro e independentes de detalhes de infraestrutura (MARTIN, 2017). A solução do *back-end* foi dividida em cinco projetos, e o *front-end* constitui uma aplicação de página única (*Single Page Application*, SPA), conforme a Figura 1.

Figura 1 – Arquitetura em camadas da aplicação AntRoute



Fonte: elaborada pelo autor (2026).

O projeto *Domain* concentra os modelos, as interfaces e os motores de otimização, não possuindo dependências externas. O projeto *Services* orquestra os casos de uso, como otimizar uma rota ou comparar algoritmos, dependendo apenas do domínio. O projeto *Infrastructure* implementa as integrações externas, isto é, o cliente HTTP do OSRM e a persistência em banco de dados. O projeto *IoC* (*Inversion of Control*) registra as dependências, e o projeto *Server* expõe a Interface de Programação de Aplicações (*Application Programming Interface*, API) e hospeda a SPA. Essa organização permitiu testar os algoritmos isoladamente e substituir a fonte de distâncias sem alterar as regras de negócio. O Quadro 1 resume as tecnologias empregadas.

Quadro 1 – Tecnologias utilizadas no desenvolvimento

Camada	Tecnologia	Finalidade
Back-end	.NET 10 / C# 14, ASP.NET Core Web API	Regras de negócio, algoritmos e API REST
Tempo real	Server-Sent Events (SSE)	Transmissão das iterações da ACO ao navegador
Persistência	Entity Framework Core 10 e SQLite	Armazenamento do módulo de geração de dados
Roteamento	OSRM (OpenStreetMap)	Matriz de distâncias rodoviárias
Geocodificação	Photon (Komoot)	Busca de endereços por texto
Front-end	Angular 18 e TypeScript 5.5	Interface de página única
Mapas e gráficos	Leaflet 1.9 e Chart.js 4.5	Exibição de rotas e gráficos comparativos
Testes	xUnit, Moq, FluentAssertions, Jasmine e Karma	Testes automatizados do back-end e do front-end
Implantação	GitHub Actions e Azure Web Apps	Integração e entrega contínuas

Fonte: elaborada pelo autor (2026).

3.3 OBTENÇÃO DA MATRIZ DE DISTÂNCIAS

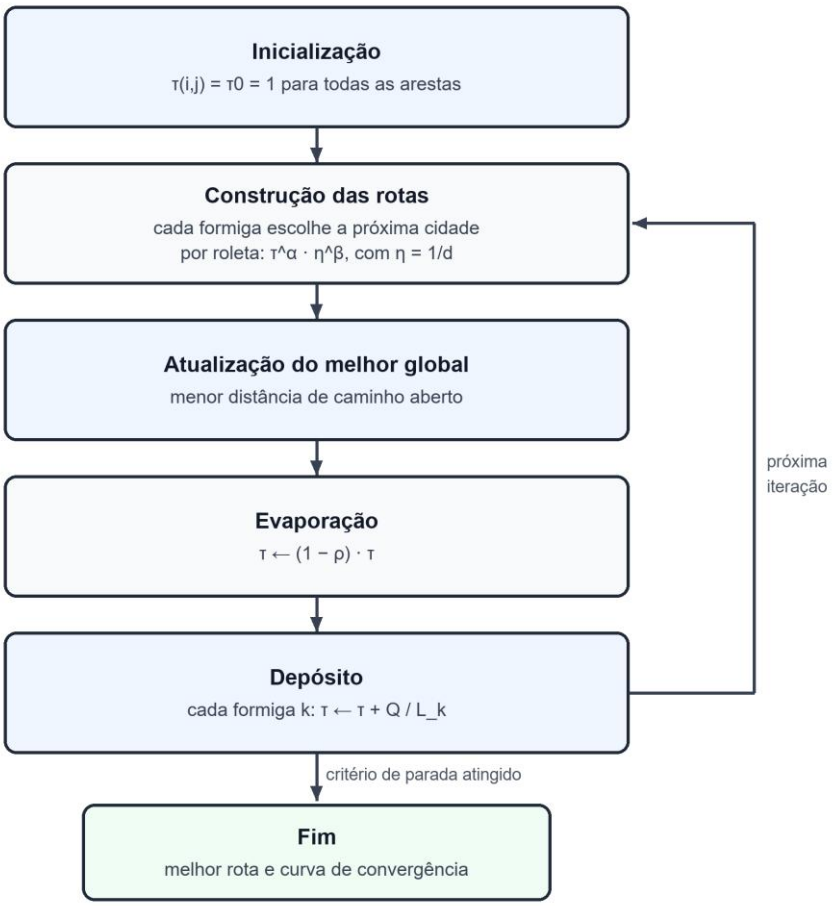
Ao receber uma solicitação de otimização, o serviço de rotas consulta o serviço *table* do OSRM com as coordenadas de todos os locais. Dois cuidados de implementação mostraram-se necessários: a API do OSRM espera as coordenadas na ordem longitude, latitude, inversa à convenção usual, e o servidor público recusa requisições sem o cabeçalho *User-Agent*. As distâncias retornadas em metros são mantidas nessa unidade durante a otimização e convertidas para quilômetros na apresentação. Como a matriz rodoviária é, em geral, assimétrica, todos os algoritmos foram implementados de modo a respeitar o sentido de cada deslocamento. Após a otimização, a geometria da rota final é solicitada ao OSRM para ser desenhada sobre as ruas no mapa.

A aplicação exige ao menos dois locais e permite, opcionalmente, fixar o ponto de partida, informando o índice do local inicial, que deve estar entre 0 e $n - 1$. Todos os algoritmos otimizam o caminho aberto definido na Equação 1.

3.4 IMPLEMENTAÇÃO DOS ALGORITMOS

A ACO foi implementada na variante *Ant System*, cujo fluxo é apresentado na Figura 2. O feromônio de todas as arestas é inicializado com $\tau_0 = 1$. Em cada iteração, cada formiga parte do local fixado pelo usuário ou, na ausência dele, de um local sorteado, e constrói sua rota escolhendo o próximo local por roleta, com pesos proporcionais a $\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}$ (Equações 3 e 4). Pontos coincidentes, com distância nula, recebem peso zero para evitar divisão por zero. Concluídas as rotas, atualiza-se o melhor caminho global, aplica-se a evaporação a toda a matriz (Equação 5) e cada formiga deposita Q/L_k, com $Q = 1$, nas arestas percorridas e em suas opostas (Equação 6). Não foram empregados elitismo, limites de feromônio ou busca local, de modo que o comportamento observado corresponde ao do AS clássico. O gerador de números aleatórios aceita uma semente opcional, o que torna as execuções reprodutíveis no módulo de geração de dados.

Figura 2 – Fluxo da Otimização por Colônia de Formigas implementada



Fonte: elaborada pelo autor (2026).

Além da melhor rota e de sua distância, cada execução registra a iteração em que a melhor solução foi encontrada, o total de rotas construídas (formigas × iterações) e a curva de convergência, isto é, a melhor distância conhecida ao fim de cada iteração. Por construção, essa curva é monotonicamente não crescente.

O Vizinho Mais Próximo parte do local fixado, ou do primeiro local, e segue sempre para o vizinho não visitado de menor distância. O 2-Opt utiliza a rota do Vizinho Mais

Próximo como solução inicial e aplica a estratégia de primeira melhoria (*first improvement*), mantendo fixo o local inicial e permitindo a alteração do local final, como convém ao caminho aberto. Como a matriz rodoviária é assimétrica, o ganho de cada movimento não pode ser avaliado apenas pelas duas arestas de fronteira: a inversão do segmento troca também o sentido de todas as arestas internas, cujo custo muda quando $d_{ij} \neq d_{ji}$. A implementação calcula, portanto, a variação exata de custo do movimento, somando às arestas de fronteira a diferença acumulada das arestas invertidas; a distância final é ainda recalculada a partir da rota resultante. Esse cuidado é necessário à correção do método: com o ganho aproximado, válido apenas para matrizes simétricas, a distância acompanhada diverge da rota efetivamente construída e um movimento e seu inverso podem parecer vantajosos indefinidamente. O Algoritmo Genético gera a população inicial com permutações aleatórias, seleciona os genitores por torneio, aplica o *Order Crossover* com dois pontos de corte, preservando o local inicial quando fixado, e a mutação por troca; a cada geração a população é inteiramente substituída, mas a melhor solução já encontrada é preservada em separado. Tal como a ACO, o Algoritmo Genético aceita uma semente opcional: na aplicação interativa ele é executado sem semente, enquanto nos experimentos a semente é derivada da semente mestre, o que torna cada execução reproduzível. Os parâmetros padrão estão no Quadro 2.

Quadro 2 – Parâmetros padrão dos algoritmos

Algoritmo	Parâmetro	Valor
ACO (Ant System)	Número de formigas (m)	20
	Iterações	100
	Peso do feromônio (α)	1,0
	Peso heurístico (β)	2,0
	Taxa de evaporação (ρ)	0,5
	Feromônio inicial (τ_0) / constante de depósito (Q)	1 / 1
Algoritmo Genético	Tamanho da população	100
	Gerações	200
	Taxa de mutação	0,02
	Tamanho do torneio	5
Vizinho Mais Próximo	—	sem parâmetros
2-Opt	Solução inicial / estratégia	NN / primeira melhoria com ganho exato

Fonte: elaborada pelo autor (2026).

Para obter a solução ótima, utilizada como referência no módulo de geração de dados, foram implementados dois métodos exatos que suportam matrizes assimétricas e o caminho aberto: força bruta com poda, aplicada a instâncias de até 10 pontos, e a programação dinâmica de Held-Karp (Equação 2), aplicada a instâncias de até 15 pontos. Cada cálculo é limitado a 60 segundos; instâncias que excedem esse tempo são descartadas.

3.5 FUNCIONALIDADES DA INTERFACE

A interface web possui três telas. Na tela principal, o usuário adiciona locais clicando no mapa ou buscando endereços pelo serviço de geocodificação Photon, cuja pesquisa é disparada após uma pausa de 400 ms na digitação para evitar requisições excessivas. É possível marcar o ponto de partida, salvar os pontos no armazenamento local do navegador e importá-los ou exportá-los em formato CSV (*Comma-Separated Values*). A partir dessa tela, o usuário executa a ACO no modo convencional ou no modo visual.

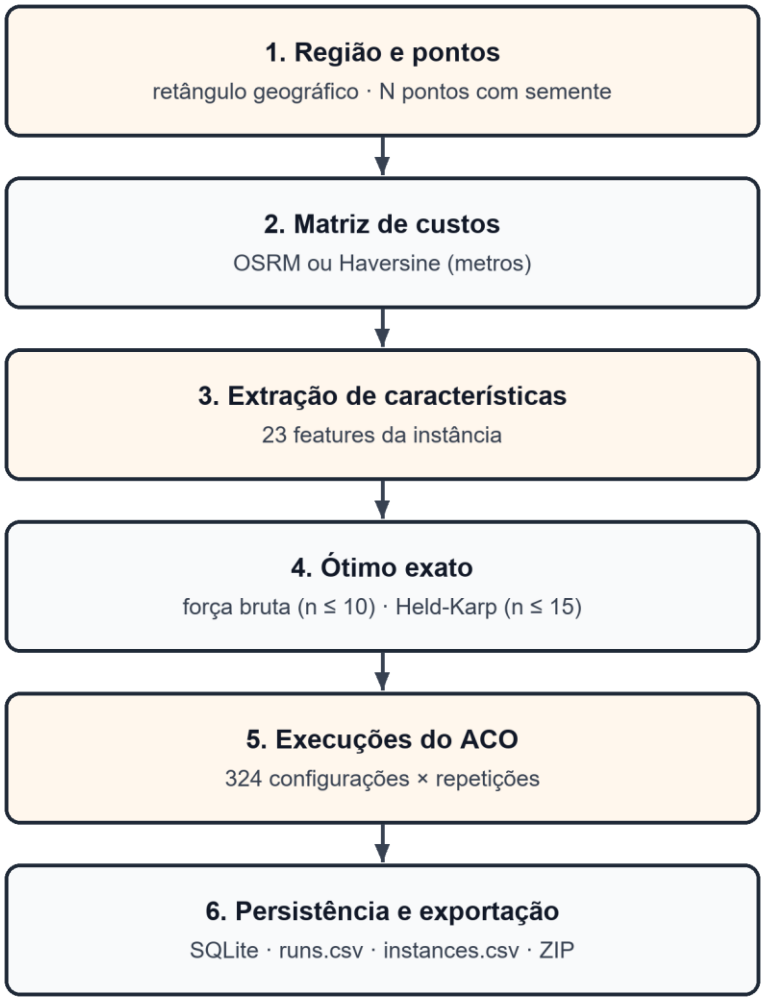
No modo visual, o servidor transmite cada iteração ao navegador por meio de *Server-Sent Events* (SSE), mecanismo padronizado do HTML em que o servidor mantém uma conexão HTTP aberta e envia eventos de texto à medida que são produzidos (WHATWG, 2026). Cada evento contém a iteração corrente, a melhor rota e a melhor distância até o momento e as rotas construídas pelas formigas, que são desenhadas sobre o mapa. Um atraso artificial de 50 ms entre eventos torna a animação perceptível ao usuário. Ao final, um evento de resultado entrega a rota otimizada com a geometria sobre as ruas.

A tela de comparação executa sequencialmente os quatro algoritmos sobre a mesma matriz de distâncias, medindo o tempo de execução de cada um, e exibe as rotas no mapa com cores distintas, além de gráficos de barras de distância e tempo. Para cada algoritmo é calculada a diferença percentual em relação à menor distância obtida entre todos (Equação 7), de modo que o melhor algoritmo apresenta 0,00%. A terceira tela corresponde ao módulo de geração de dados, descrito a seguir.

3.6 MÓDULO DE GERAÇÃO DE DADOS

Com o objetivo de produzir dados para o estudo dos hiperparâmetros da ACO, foi desenvolvido um módulo que executa tarefas (*jobs*) em segundo plano, uma de cada vez, a partir de uma fila. Cada tarefa segue o fluxo da Figura 3.

Figura 3 – Fluxo de processamento de uma instância no módulo de geração de dados



Fonte: elaborada pelo autor (2026).

O usuário define uma região retangular no mapa, a lista de tamanhos de instância desejados, o número de instâncias a sortear para cada tamanho, a fonte de distâncias (OSRM ou Haversine), a grade de hiperparâmetros e o número de repetições por configuração. Uma tarefa cobre, assim, vários tamanhos ao mesmo tempo: a lista 5, 6, …, 15 com dez instâncias por tamanho produz 110 conjuntos de pontos independentes. Para cada instância, são sorteados pontos uniformemente dentro da região e calculada a matriz de custos, arredondada ao milímetro; caso algum par de pontos seja inalcançável pela malha viária, a instância é sorteada novamente, até três tentativas. Em seguida, são extraídas as características da instância (Quadro 4), calcula-se o ótimo exato e executa-se cada configuração da grade o número de vezes definido.

Antes de registrar a instância, verifica-se um invariante do problema: como o Vizinho Mais Próximo e o 2-Opt resolvem o mesmo caminho aberto sobre a mesma matriz, nenhum deles pode apresentar distância inferior ao ótimo exato. A instância cuja referência heurística viole essa condição é marcada como falha, em vez de ingressar silenciosamente no conjunto de dados. A verificação detectou, durante o desenvolvimento, o erro de avaliação do ganho do 2-Opt em matrizes assimétricas

descrito na seção 3.4, que havia comprometido um terço das instâncias de uma geração anterior.

A reprodutibilidade foi tratada como requisito. A partir de uma semente mestre da tarefa, as sementes de cada instância e de cada execução são derivadas de forma determinística pela função de *hash* FNV-1a, combinando o identificador da instância, a chave da configuração e o número da repetição. Assim, a mesma tarefa, executada novamente, produz exatamente as mesmas instâncias e os mesmos resultados. A grade padrão, apresentada no Quadro 3, combina 3 × 3 × 3 × 4 × 3 = 324 configurações.

Quadro 3 – Grade padrão de hiperparâmetros da ACO

Hiperparâmetro	Símbolo	Valores avaliados
Peso do feromônio	α	0,5; 1; 2
Peso heurístico	β	1; 2; 5
Número de formigas	m	10; 20; 40
Taxa de evaporação	ρ	0,1; 0,3; 0,5; 0,7
Número de iterações	—	50; 100; 200

Fonte: elaborada pelo autor (2026).

As 23 características extraídas, sintetizadas no Quadro 4, descrevem o tamanho e a geometria da instância, a distribuição dos custos e a dificuldade relativa percebida por heurísticas simples. O índice de Clark-Evans compara a distância média ao vizinho mais próximo com a esperada em uma distribuição aleatória de mesma densidade, indicando agrupamento (valores menores que 1) ou regularidade (maiores que 1) (CLARK; EVANS, 1954). O índice de assimetria mede a diferença média entre d_{ij} e d_{ji}, e o índice de desvio (*detour index*) relaciona o custo rodoviário médio à distância em linha reta.

Quadro 4 – Características extraídas de cada instância

Grupo	Características
Tamanho e região	número de pontos; largura, altura e área da região (km); densidade de pontos (pontos/km²)
Distribuição espacial	média e desvio-padrão da distância ao centroide; distância média ao vizinho mais próximo; índice de Clark-Evans
Matriz de custos	média, desvio-padrão, mínimo, máximo e mediana; coeficiente de variação; média e desvio-padrão do custo ao vizinho mais próximo
Estrutura da malha	índice de assimetria; índice de desvio (custo / distância em linha reta)
Referências heurísticas	distância do Vizinho Mais Próximo e do 2-Opt; gap de ambos em relação ao ótimo (%)

Fonte: elaborada pelo autor (2026).

Os resultados são persistidos em banco SQLite por meio do Entity Framework Core, de forma atômica por instância, em três tabelas: tarefas, instâncias e execuções. Ao reiniciar o servidor, tarefas interrompidas podem ser retomadas, reprocessando apenas as instâncias pendentes ou com falha. Limites configuráveis protegem o servidor de tarefas excessivas: até 200 instâncias por tarefa, 30 repetições e 200 mil

execuções. Os dados podem ser exportados em três arquivos CSV: o de execuções, principal tabela para o treinamento de modelos, com uma linha por execução contendo as características da instância, os hiperparâmetros e as métricas obtidas, tendo o identificador e o número de pontos da instância entre as colunas-chave; o de instâncias, com coordenadas, matriz e rota ótima; e o de resumo por configuração, com média, desvio-padrão, mínimo e máximo do gap e taxa de acerto do ótimo. A exportação em formato ZIP agrupa os três arquivos e um arquivo de metadados.

3.7 PROTOCOLO EXPERIMENTAL E MÉTRICAS

A principal métrica de qualidade é o gap relativo, definido na Equação 7, em que D é a distância obtida pelo algoritmo e D^* é a distância de referência. Na tela de comparação, D^* corresponde à menor distância obtida entre os quatro algoritmos; no módulo de geração de dados, ao ótimo exato. Considera-se que uma execução atingiu o ótimo quando sua diferença absoluta para D^* não excede $10^{-9} \cdot \max(1, D^*)$.

$$\text{Gap}\ (\%) = (D - D^*) / D^* \times 100 \qquad (7)$$

Também são registrados o tempo de execução, medido em milissegundos pelo relógio de parede, a iteração em que a melhor solução foi encontrada e a taxa de acerto do ótimo, isto é, a proporção de repetições em que o ótimo foi atingido. Os experimentos foram organizados em dois conjuntos:

a) **comparação entre algoritmos:** 10 instâncias para cada tamanho de 10, 15, 30 e 50 pontos, sorteadas na região da Grande Vitória (ES) com distâncias rodoviárias do OSRM, executando os quatro algoritmos sobre a mesma matriz de custos com os parâmetros padrão do Quadro 2 e 10 repetições dos métodos estocásticos, o que totaliza 40 instâncias e 880 execuções. Em 10 e 15 pontos a referência do gap é o ótimo exato; em 30 e 50, a menor distância obtida na instância;

b) **sensibilidade aos hiperparâmetros:** seis tarefas do módulo de geração de dados, distribuídas em três escalas espaciais da Grande Vitória (ES) — o centro da ilha, com cerca de 14 km²; a ilha completa, com cerca de 88 km²; e a região metropolitana, com cerca de 217 km² —, duas tarefas por escala. Cada tarefa cobriu os tamanhos de 5 a 15 pontos, com cinco instâncias por tamanho, perfazendo 330 instâncias, todas com distâncias rodoviárias do OSRM e ótimo exato calculado. Sobre cada instância foi executada a grade completa de 324 configurações com 3 repetições, totalizando 320.760 execuções.

A escolha de três escalas espaciais, em vez de uma única região, teve por finalidade diversificar as características das instâncias: a área da região varia de 13,9 km² a 217,4 km², o índice de desvio de 1,28 a 5,69 e o índice de Clark-Evans de 0,52 a 1,84, cobrindo desde conjuntos agrupados até conjuntos de distribuição regular. As sementes mestras utilizadas foram 20250101, 20250102, 20250202, 20250203, 20250303 e 20250304, de modo que todo o conjunto é reproduzível.

Os experimentos foram executados em um computador com processador AMD Ryzen 5 5500 (6 núcleos e 12 *threads*), 16 GB de memória RAM e Windows 11 Pro, sob o .NET 10, em compilação *Release* da aplicação. A geração completa do conjunto de dados consumiu aproximadamente 1 hora e 13 minutos.

3.8 VERIFICAÇÃO E IMPLANTAÇÃO

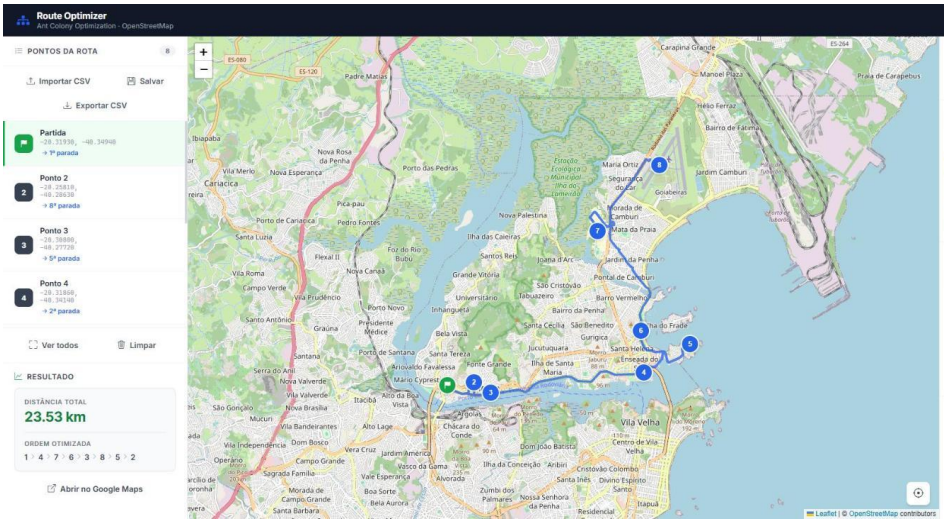
A correção da implementação foi verificada por testes automatizados. No *back-end*, 105 métodos de teste escritos com xUnit, Moq e FluentAssertions cobrem os motores de otimização, por exemplo verificando que a ACO nunca piora a melhor distância entre iterações, que os métodos exatos coincidem com a enumeração completa e que, em matrizes assimétricas sorteadas aleatoriamente, a distância relatada pelo 2-Opt corresponde exatamente ao custo da rota que ele devolve, além dos serviços, do módulo de geração de dados e dos controladores da API. No *front-end*, 176 casos de teste em Jasmine e Karma cobrem componentes e serviços, incluindo a leitura do fluxo SSE. Um fluxo de integração contínua no GitHub Actions executa ambos os conjuntos a cada alteração e, quando o código é integrado ao ramo principal, publica a aplicação no Microsoft Azure Web Apps, com autenticação federada e sem segredos de longa duração.

4 RESULTADOS E DISCUSSÃO

4.1 A APLICAÇÃO ANTROUTE

A Figura 4 apresenta a tela principal da aplicação com um exemplo real: oito locais da ilha de Vitória e do entorno — do Parque Moscoso, fixado como partida, ao aeroporto —, importados de um arquivo CSV e otimizados pela ACO com os parâmetros padrão. A rota obtida tem 23,53 km e foi calculada em 6 ms, sem contar o tempo de consulta ao OSRM; a ordem de visita resultante, mostrada no painel esquerdo, difere da ordem de entrada dos pontos. A rota é desenhada sobre a malha viária, e não em linha reta, porque a geometria do trajeto é solicitada ao OSRM após a otimização.

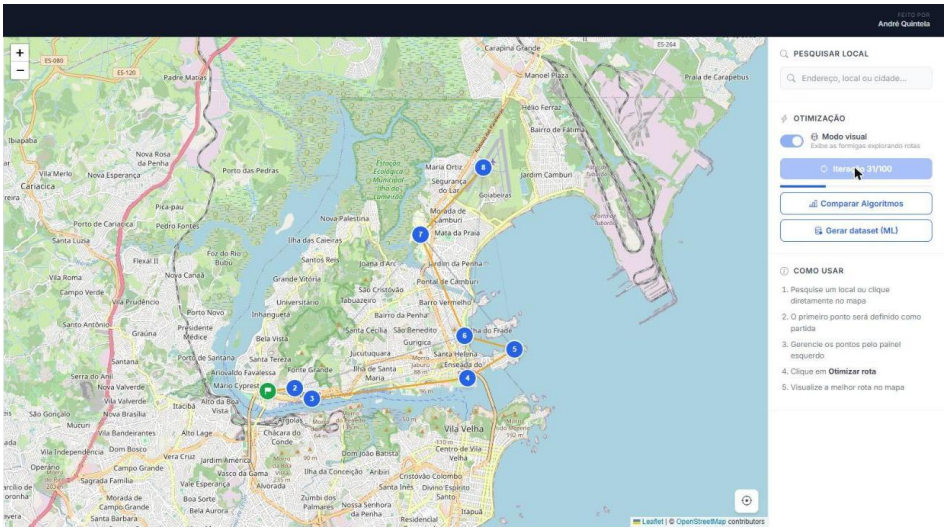
Figura 4 – Tela principal com rota otimizada pela ACO



Fonte: elaborada pelo autor (2026).

O modo visual, ilustrado na Figura 5, mostrou-se útil para compreender o comportamento da colônia: nas primeiras iterações, as rotas das formigas são dispersas e cobrem grande parte das arestas possíveis; à medida que o feromônio se acumula, as rotas passam a compartilhar a maior parte das arestas e convergem para a melhor solução. Esse efeito torna perceptível o equilíbrio entre exploração e intensificação discutido na seção 2.5 e confere à ferramenta potencial didático.

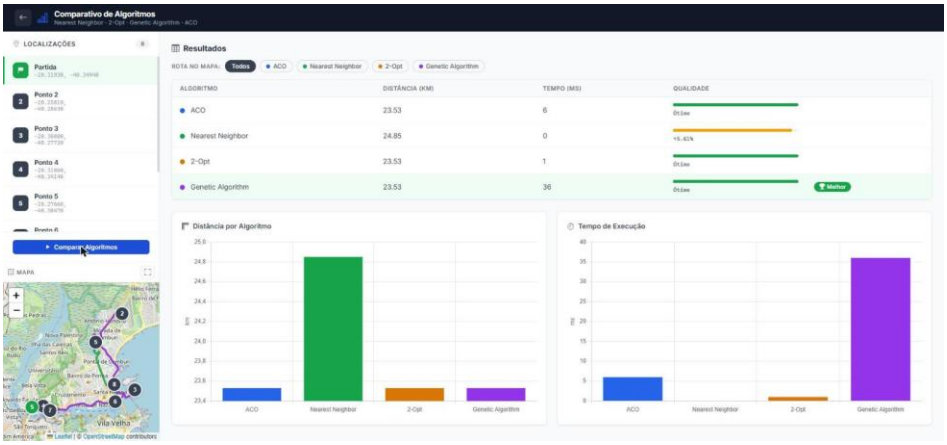
Figura 5 – Modo visual: rotas das formigas durante a execução



Fonte: elaborada pelo autor (2026).

A tela de comparação (Figura 6) reúne as rotas dos quatro algoritmos, os gráficos de distância e tempo e o gap de cada um em relação ao melhor resultado. Para o mesmo exemplo de oito pontos, a ACO, o 2-Opt e o Algoritmo Genético encontraram a mesma rota de 23,53 km, em 6, 1 e 36 ms respectivamente, enquanto o Vizinho Mais Próximo obteve 24,85 km, 5,61% acima dos demais, em tempo inferior a 1 ms. O exemplo ilustra o que a seção 4.2 mostra em escala: em instâncias pequenas os três métodos mais elaborados convergem para a mesma solução, e a diferença entre eles aparece apenas no tempo.

Figura 6 – Tela de comparação entre algoritmos



Fonte: elaborada pelo autor (2026).

4.2 COMPARAÇÃO ENTRE ALGORITMOS

A Tabela 1 apresenta, para cada tamanho de instância, a média da distância, do gap e do tempo de execução de cada algoritmo, conforme o protocolo da seção 3.7.

Tabela 1 – Desempenho médio dos algoritmos por tamanho de instância

n	Algoritmo	Distância (km)	Gap (%)	Tempo (ms)
10	ACO	62,480	0,03	3,438

n	Algoritmo	Distância (km)	Gap (%)	Tempo (ms)
	Vizinho Mais Próximo	67,394	8,25	0,014
	2-Opt	62,746	0,40	0,014
	Algoritmo Genético	62,867	0,64	12,810
15	ACO	83,143	2,22	6,244
	Vizinho Mais Próximo	89,720	10,21	0,005
	2-Opt	84,239	3,53	0,009
	Algoritmo Genético	83,354	2,49	14,761
30	ACO	119,258	3,15	25,663
	Vizinho Mais Próximo	128,114	11,04	0,020
	2-Opt	121,965	5,54	0,062
	Algoritmo Genético	131,319	13,70	24,995
50	ACO	159,140	2,69	72,919
	Vizinho Mais Próximo	173,985	12,24	0,007
	2-Opt	162,614	5,03	0,564
	Algoritmo Genético	218,757	41,46	42,089

Fonte: elaborada pelo autor (2026). Nota: médias sobre 10 instâncias por tamanho; ACO e Algoritmo Genético com 10 repetições cada. O gap é relativo ao ótimo exato em 10 e 15 pontos e, em 30 e 50 pontos, à menor distância obtida na instância. Tempos abaixo de 0,1 ms estão no limite de resolução do relógio.

Os resultados confirmam o comportamento previsto pela literatura. O Vizinho Mais Próximo é, de longe, o mais rápido, resolvendo qualquer um dos tamanhos em menos de 0,02 ms, mas apresenta os maiores gaps, que crescem de 8,25% em instâncias de 10 pontos para 12,24% em instâncias de 50, exatamente o efeito das arestas longas deixadas ao final da rota descrito por Rosenkrantz, Stearns e Lewis (1977). O 2-Opt reduz esse gap a uma fração do valor — de 8,25% para 0,40% em 10 pontos e de 12,24% para 5,03% em 50 — a um custo desprezível, no máximo 0,56 ms, o que o confirma como o melhor compromisso entre qualidade e tempo quando o orçamento computacional é apertado.

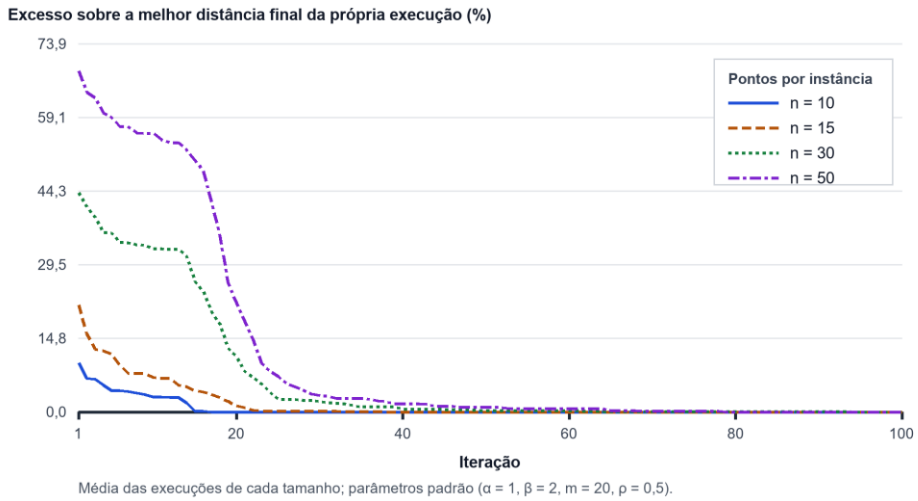
A ACO obteve a menor distância média nos quatro tamanhos avaliados e venceu 29 das 40 instâncias, contra 7 do 2-Opt e 4 do Algoritmo Genético. Seu gap vai de 0,03% em instâncias de 10 pontos, praticamente o ótimo, a 3,15% em instâncias de 30, mantendo-se abaixo do 2-Opt em toda a faixa. O custo é o tempo: 72,9 ms em instâncias de 50 pontos, cerca de 130 vezes o do 2-Opt, para ganhar 2,34 pontos percentuais de gap. Como o tempo da ACO é proporcional ao número de rotas construídas, formigas × iterações, esse custo é previsível e ajustável. O Algoritmo Genético acompanha a ACO nas instâncias pequenas, com gaps de 0,64% e 2,49% em 10 e 15 pontos, mas degrada rapidamente à medida que o espaço de busca cresce: 13,70% em 30 pontos e 41,46% em 50, com tempo da mesma ordem do da ACO. Com população e número de gerações fixos, a cobertura do espaço de permutações diminui à medida que n aumenta, e o método passa a ser dominado até pela busca local.

Quanto à variabilidade entre repetições, a ACO mostrou-se bem mais estável que o Algoritmo Genético: o coeficiente de variação médio das distâncias dentro de uma mesma instância ficou entre 0,06% e 1,31% para a ACO, contra 1,34% a 5,69% para o Algoritmo Genético, com a dispersão de ambos crescendo com o tamanho da instância. Tomando a melhor das dez repetições em vez da média, o gap da ACO cai para 0,00% em 10 pontos e permanece abaixo de 0,85% nos demais tamanhos, o que indica que boa parte da diferença para o ótimo se deve à variabilidade entre execuções e não a um limite da configuração utilizada. O Vizinho Mais Próximo e o 2-Opt são determinísticos e, portanto, não apresentam dispersão.

4.3 CONVERGÊNCIA DA ACO

A Figura 7 mostra a convergência da ACO com os parâmetros padrão nas instâncias do experimento de comparação. Para tornar os quatro tamanhos comparáveis na mesma escala, cada curva foi normalizada como o excesso percentual da melhor distância conhecida ao final de cada iteração sobre a melhor distância que aquela mesma execução alcançou ao final.

Figura 7 – Convergência da ACO por tamanho de instância



Fonte: elaborada pelo autor (2026).

A forma das curvas é a mesma nos quatro tamanhos, mas a escala e o ritmo mudam: em instâncias de 10 pontos a primeira iteração já está a cerca de 10% da solução final e a curva se estabiliza por volta da iteração 15, enquanto em instâncias de 50 pontos o ponto de partida está a cerca de 69% e são necessárias aproximadamente 25 iterações para chegar perto do valor final. A iteração média em que a melhor solução foi encontrada acompanha esse crescimento: 9,7 em instâncias de 10 pontos, 21,1 em 15, 43,4 em 30 e 53,9 em 50.

Nas instâncias menores do experimento de sensibilidade, a convergência é ainda mais precoce. Sobre as 320.760 execuções, a melhor solução foi encontrada muito cedo: a mediana da iteração em que isso ocorreu é 5, qualquer que seja o limite de iterações configurado, e a média é de 10,7 iterações quando o limite é 50, 16,2 quando é 100 e 20,9 quando é 200. Em outras palavras, nas execuções com 100 iterações apenas cerca de 16% delas contribuíram, em média, para o resultado final. O aumento do limite ainda assim reduz o gap médio, de 1,811% com 50 iterações para 1,360%

com 100 e 1,197% com 200, o que indica que o ganho das iterações adicionais vem de uma minoria de execuções difíceis, e não de uma melhora contínua em todas elas. Os dois experimentos, em conjunto, mostram que o orçamento de iterações necessário cresce com o tamanho da instância: as 100 iterações padrão são largamente suficientes até 15 pontos e passam a ser efetivamente utilizadas a partir de 30. Uma convergência muito precoce sugere estagnação, situação em que todas as formigas passam a construir a mesma rota, comportamento conhecido do AS com evaporação baixa e que motivou variantes como o MMAS (STÜTZLE; HOOS, 2000).

4.4 SENSIBILIDADE AOS HIPERPARÂMETROS

Considerado todo o conjunto, o gap médio da ACO em relação ao ótimo exato foi de 1,456%, com mediana nula, nonagésimo percentil de 4,596% e máximo de 55,277%; 67,6% das execuções alcançaram o ótimo. A dificuldade, porém, não se distribui igualmente entre os tamanhos de instância, como mostra a Tabela 2.

Tabela 2 – Gap da ACO e taxa de acerto do ótimo por número de pontos

n	Instâncias	Gap médio (%)	Mediana (%)	Percentil 90 (%)	Taxa de ótimo (%)
5	30	0,004	0,000	0,000	99,9
6	30	0,058	0,000	0,000	98,9
7	30	0,018	0,000	0,000	98,1
8	30	0,271	0,000	0,000	92,8
9	30	0,994	0,000	2,280	78,1
10	30	1,856	0,000	2,235	75,3
11	30	2,140	0,000	10,917	61,0
12	30	1,385	0,000	4,567	55,6
13	30	2,225	1,061	6,627	35,8
14	30	3,556	1,353	10,279	27,0
15	30	3,510	1,518	9,950	20,9

Fonte: elaborada pelo autor (2026). Nota: 972 execuções por instância (324 configurações × 3 repetições).

A taxa de acerto cai de forma praticamente monotônica, de 99,9% em instâncias de 5 pontos para 20,9% em instâncias de 15, o que confirma que o espaço de busca cresce mais rápido do que a capacidade do *Ant System* de percorrê-lo com os orçamentos avaliados. A consequência metodológica é relevante para a etapa seguinte da pesquisa: em instâncias de até 7 pontos o problema é praticamente trivial e quase toda configuração encontra o ótimo, de modo que esses casos pouco informam um modelo de recomendação.

A Tabela 3 lista as dez configurações com menor gap médio entre as 324 avaliadas.

Tabela 3 – As dez configurações da ACO com menor gap médio em relação ao ótimo

α	β	m	ρ	Iter.	Gap médio (%)	Desvio (%)	Taxa de ótimo (%)	Tempo médio (ms)
0,5	2	40	0,3	200	0,566	3,021	86,3	33,9

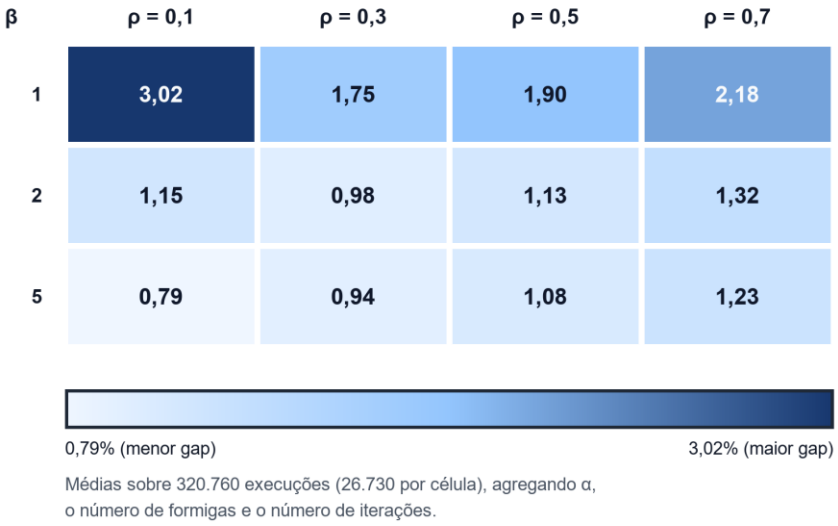
α	β	m	ρ	Iter.	Gap médio (%)	Desvio (%)	Taxa de ótimo (%)	Tempo médio (ms)
0,5	2	40	0,5	200	0,573	3,022	86,0	34,0
0,5	2	40	0,1	200	0,585	3,040	84,8	34,3
0,5	2	40	0,7	200	0,599	3,030	85,7	34,2
1	2	40	0,1	200	0,602	3,041	85,8	30,3
0,5	2	20	0,3	200	0,620	3,053	83,6	17,2
0,5	5	40	0,1	200	0,621	3,045	83,4	33,9
0,5	5	40	0,1	100	0,637	3,049	82,8	16,9
1	1	40	0,1	200	0,640	3,071	83,2	21,8
0,5	2	40	0,5	100	0,647	3,078	83,6	17,0

Fonte: elaborada pelo autor (2026). Nota: médias sobre 330 instâncias e 3 repetições, ou seja, 990 execuções por configuração.

As dez melhores configurações compartilham um padrão nítido: peso do feromônio baixo (α de 0,5 ou 1), peso heurístico igual ou superior a 2, população grande (20 ou 40 formigas) e orçamento alto de iterações. A configuração padrão da aplicação (α = 1, β = 2, m = 20, ρ = 0,5 e 100 iterações) ocupa a 139ª posição, com gap médio de 1,045% e 70,6% de acerto do ótimo, consumindo 7,6 ms por execução; a melhor configuração reduz o gap para 0,566% e eleva o acerto a 86,3%, porém a um custo de 33,9 ms, cerca de 4,5 vezes maior. A pior configuração avaliada (α = 0,5, β = 1, m = 10, ρ = 0,1 e 50 iterações) apresenta gap médio de 7,053%, mais de doze vezes o da melhor, executando em 2,0 ms. Há, portanto, um compromisso explícito entre qualidade e tempo, e é justamente esse compromisso que um modelo de recomendação por instância poderia explorar.

Para analisar o efeito isolado de cada hiperparâmetro, a Figura 8 apresenta o gap médio para cada combinação de β e ρ, agregando os demais parâmetros.

Figura 8 – Gap médio (%) da ACO por combinação de β e ρ



Fonte: elaborada pelo autor (2026).

Os dados confirmam a tendência apontada pela literatura, segundo a qual valores de β entre 2 e 5 favorecem o desempenho do AS no PCV, por darem peso suficiente à informação heurística, e segundo a qual a taxa de evaporação controla o compromisso entre convergência e diversidade (DORIGO; STÜTZLE, 2004). O peso heurístico é o hiperparâmetro mais influente: o gap médio cai de 2,211% com β = 1 para 1,146% com β = 2 e 1,012% com β = 5. A taxa de evaporação tem efeito menor e não monotônico, com mínimo em ρ = 0,3 (1,223%) e piora tanto para ρ = 0,1 (1,655%) quanto para ρ = 0,7 (1,576%); a interação entre os dois, visível na Figura 8, mostra que a evaporação baixa só é vantajosa quando acompanhada de peso heurístico alto. O número de formigas atua de forma monotônica, com gaps de 1,878%, 1,395% e 1,096% para 10, 20 e 40 formigas, assim como o número de iterações. Já o peso do feromônio apresenta ótimo intermediário, com 1,303% em α = 1, contra 1,414% em α = 0,5 e 1,651% em α = 2, o que é compatível com a estagnação esperada quando o feromônio domina a decisão.

A análise da melhor configuração por instância, que seria o rótulo natural de um modelo de classificação, revelou uma dificuldade do conjunto: os empates são maciços. Em instâncias de 5 pontos, a mediana é de 324 configurações empatadas no mesmo gap mínimo, isto é, todas; a mediana cai para 116 em instâncias de 12 pontos e para 13 em instâncias de 15. Em apenas 16 das 330 instâncias existe uma única configuração vencedora. Em contrapartida, a composição do conjunto ótimo varia: considerando as 272 instâncias em que nem todas as configurações empatam, a configuração mais robusta (α = 0,5, β = 2, m = 40, ρ = 0,3 e 200 iterações) pertence ao conjunto de melhores em 84,2% dos casos, e as 324 configurações aparecem nesse conjunto em pelo menos uma instância. Esse resultado tem consequência direta para o trabalho futuro: a formulação adequada não é classificar a configuração vencedora, mal definida sob tantos empates, e sim estimar o gap esperado a partir das características da instância e dos hiperparâmetros, escolhendo na inferência a configuração de menor gap previsto.

4.5 DISCUSSÃO E LIMITAÇÕES

Os resultados devem ser interpretados à luz de algumas limitações. Primeiro, a comparação com o ótimo exato restringe-se a instâncias de até 15 pontos, em razão da complexidade exponencial do algoritmo de Held-Karp; para instâncias maiores, a referência é a melhor solução entre os algoritmos, que não é necessariamente ótima. Segundo, a ACO implementada é o *Ant System* original, sem elitismo nem busca local, o que a coloca em desvantagem frente a variantes mais recentes, como o ACS e o MMAS, que costumam obter resultados superiores no PCV (DORIGO; STÜTZLE, 2004). Terceiro, o experimento de sensibilidade utilizou o servidor público do OSRM, que impõe limites de uso e não oferece garantia de disponibilidade; foi necessário inserir um intervalo de um segundo entre as requisições, o que limita a escala viável do conjunto de dados. Quarto, todas as instâncias provêm de uma única região metropolitana, de modo que a transferência das conclusões para malhas viárias com outra topologia, como as de cidades planejadas ou de áreas rurais, permanece por verificar. Quinto, o elevado número de empates no gap mínimo, discutido na seção 4.4, restringe o uso do conjunto para a classificação direta da melhor configuração. Por fim, o Algoritmo Genético foi avaliado com os parâmetros padrão do Quadro 2, que não foram ajustados por tamanho de instância; o desempenho relatado para as instâncias maiores reflete essa escolha, e não um limite intrínseco do método.

Cabe ainda registrar uma lição metodológica do desenvolvimento. A primeira geração do conjunto de dados foi descartada porque o 2-Opt avaliava o ganho dos movimentos pela fórmula válida apenas para matrizes simétricas, o que, sobre as distâncias rodoviárias assimétricas do OSRM, produziu distâncias inferiores ao ótimo exato em 34% das instâncias e, em alguns casos, impediu a terminação da busca local. O episódio evidencia a importância de verificar invariantes do problema, e não apenas a ausência de exceções, em experimentos computacionais; a verificação descrita na seção 3.6 foi incorporada ao módulo justamente para impedir que valores inconsistentes voltem a ingressar no conjunto de dados.

Os resultados de sensibilidade convergem com Dorigo e Stützle (2004) quanto ao papel dominante da informação heurística e quanto ao risco de estagnação associado ao peso excessivo do feromônio, observado aqui em $\alpha = 2$. A convergência precoce medida na seção 4.3 nas instâncias pequenas, com mediana de cinco iterações para a melhor solução, é igualmente compatível com a motivação do MMAS (STÜTZLE; HOOS, 2000), que introduz limites de feromônio precisamente para retardar esse efeito. Na comparação entre algoritmos, a ordem observada entre o Vizinho Mais Próximo e o 2-Opt reproduz o que a literatura relata desde Croes (1958) e Rosenkrantz, Stearns e Lewis (1977): a construção gulosa é barata e sistematicamente pior, e uma única rodada de busca local recupera a maior parte da diferença. A divergência está no Algoritmo Genético, que a literatura costuma apresentar como competitivo com a ACO no PCV: aqui ele só o foi até 15 pontos, degradando-se a 41,46% de gap em 50. A diferença é de configuração, não de método — população e gerações fixas em 100 e 200 cobrem uma fração cada vez menor do espaço de permutações à medida que n cresce —, o que reforça, do lado do Algoritmo Genético, a mesma tese que motiva este trabalho do lado da ACO: o desempenho de uma metaheurística é inseparável do ajuste de seus hiperparâmetros à instância.

5 CONSIDERAÇÕES FINAIS

Este trabalho teve como objetivo desenvolver uma aplicação web para a otimização de rotas por colônia de formigas sobre distâncias rodoviárias reais, comparar seu desempenho com o de outros algoritmos e produzir uma base de dados experimental sobre seus hiperparâmetros. O objetivo foi alcançado com a construção da aplicação AntRoute, que integra o cadastro de locais em mapa interativo, a obtenção de distâncias pelo OSRM, a execução de quatro algoritmos de otimização e a visualização, em tempo real, da convergência da colônia.

Quanto aos objetivos específicos, foram implementados a ACO na variante *Ant System*, o Vizinho Mais Próximo, o 2-Opt e um Algoritmo Genético, todos adaptados ao caminho aberto e a matrizes assimétricas, além de métodos exatos de força bruta e Held-Karp. A interface permite comparar os algoritmos lado a lado, por meio da distância, do gap e do tempo de execução. Na comparação realizada, a ACO obteve a menor distância média nos quatro tamanhos avaliados e venceu 29 das 40 instâncias, embora ao custo do maior tempo de execução; o 2-Opt mostrou-se a alternativa mais eficiente quando o tempo é crítico, alcançando a maior parte do ganho a uma fração do custo.

A principal contribuição acadêmica é o módulo de geração de dados, que produz, de forma reprodutível, um conjunto de instâncias descritas por 23 características e associadas ao desempenho de 324 configurações da ACO em relação ao ótimo exato. Os dados produzidos mostram que a escolha dos hiperparâmetros é relevante — o gap médio varia de 0,566% a 7,053% entre as 324 configurações avaliadas, e a configuração padrão da aplicação está longe da melhor — e que essa escolha envolve um compromisso explícito com o tempo de execução, condições que justificam a recomendação automática por instância. Esse conjunto constitui o insumo necessário para a etapa seguinte da pesquisa.

Como trabalhos futuros, sugere-se: treinar modelos de aprendizado de máquina sobre os dados gerados para recomendar automaticamente a configuração da ACO a partir das características da instância, avaliando o ganho em relação à configuração padrão; implementar as variantes ACS e MMAS e a combinação da ACO com a busca local 2-Opt; ampliar o estudo para instâncias maiores, utilizando como referência soluções de resolvedores de referência; e estender a aplicação para o problema de roteamento de veículos, que considera múltiplos veículos, capacidades e janelas de tempo, aproximando a ferramenta das necessidades logísticas reais.

REFERÊNCIAS

APPLEGATE, D. L.; BIXBY, R. E.; CHVÁTAL, V.; COOK, W. J. **The traveling salesman problem**: a computational study. Princeton: Princeton University Press, 2006.

BIRATTARI, M.; STÜTZLE, T.; PAQUETE, L.; VARRENTRAPP, K. A racing algorithm for configuring metaheuristics. In: GENETIC AND EVOLUTIONARY COMPUTATION CONFERENCE, 2002, New York. **Proceedings** [...]. San Francisco: Morgan Kaufmann, 2002. p. 11-18.

BLUM, C.; ROLI, A. Metaheuristics in combinatorial optimization: overview and conceptual comparison. **ACM Computing Surveys**, New York, v. 35, n. 3, p. 268-308, 2003.

CLARK, P. J.; EVANS, F. C. Distance to nearest neighbor as a measure of spatial relationships in populations. **Ecology**, Washington, v. 35, n. 4, p. 445-453, 1954.

CROES, G. A. A method for solving traveling-salesman problems. **Operations Research**, Baltimore, v. 6, n. 6, p. 791-812, 1958.

DAVIS, L. Applying adaptive algorithms to epistatic domains. In: INTERNATIONAL JOINT CONFERENCE ON ARTIFICIAL INTELLIGENCE, 9., 1985, Los Angeles. **Proceedings** [...]. San Francisco: Morgan Kaufmann, 1985. p. 162-164.

DENEUBOURG, J.-L.; ARON, S.; GOSS, S.; PASTEELS, J. M. The self-organizing exploratory pattern of the Argentine ant. **Journal of Insect Behavior**, New York, v. 3, n. 2, p. 159-168, 1990.

DORIGO, M.; GAMBARDELLA, L. M. Ant colony system: a cooperative learning approach to the traveling salesman problem. **IEEE Transactions on Evolutionary Computation**, Piscataway, v. 1, n. 1, p. 53-66, 1997.

DORIGO, M.; MANIEZZO, V.; COLORNI, A. Ant system: optimization by a colony of cooperating agents. **IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)**, Piscataway, v. 26, n. 1, p. 29-41, 1996.

DORIGO, M.; STÜTZLE, T. **Ant colony optimization**. Cambridge: MIT Press, 2004.

GAREY, M. R.; JOHNSON, D. S. **Computers and intractability**: a guide to the theory of NP-completeness. San Francisco: W. H. Freeman, 1979.

GOLDBERG, D. E. **Genetic algorithms in search, optimization, and machine learning**. Reading: Addison-Wesley, 1989.

HELD, M.; KARP, R. M. A dynamic programming approach to sequencing problems. **Journal of the Society for Industrial and Applied Mathematics**, Philadelphia, v. 10, n. 1, p. 196-210, 1962.

HOLLAND, J. H. **Adaptation in natural and artificial systems**. Ann Arbor: University of Michigan Press, 1975.

HUTTER, F.; HOOS, H. H.; LEYTON-BROWN, K. Sequential model-based optimization for general algorithm configuration. In: INTERNATIONAL CONFERENCE ON LEARNING AND INTELLIGENT OPTIMIZATION, 5., 2011, Rome. **Proceedings** [...]. Berlin: Springer, 2011. p. 507-523. (Lecture Notes in Computer Science, v. 6683).

LAPORTE, G. The traveling salesman problem: an overview of exact and approximate algorithms. **European Journal of Operational Research**, Amsterdam, v. 59, n. 2, p. 231-247, 1992.

LIN, S.; KERNIGHAN, B. W. An effective heuristic algorithm for the traveling-salesman problem. **Operations Research**, Baltimore, v. 21, n. 2, p. 498-516, 1973.

LUXEN, D.; VETTER, C. Real-time routing with OpenStreetMap data. In: ACM SIGSPATIAL INTERNATIONAL CONFERENCE ON ADVANCES IN GEOGRAPHIC INFORMATION SYSTEMS, 19., 2011, Chicago. **Proceedings** [...]. New York: ACM, 2011. p. 513-516.

MARTIN, R. C. **Clean architecture**: a craftsman's guide to software structure and design. Boston: Prentice Hall, 2017.

RICE, J. R. The algorithm selection problem. **Advances in Computers**, New York, v. 15, p. 65-118, 1976.

ROSENKRANTZ, D. J.; STEARNS, R. E.; LEWIS, P. M. An analysis of several heuristics for the traveling salesman problem. **SIAM Journal on Computing**, Philadelphia, v. 6, n. 3, p. 563-581, 1977.

SINNOTT, R. W. Virtues of the haversine. **Sky and Telescope**, Cambridge, v. 68, n. 2, p. 159, 1984.

SMITH-MILES, K. A. Cross-disciplinary perspectives on meta-learning for algorithm selection. **ACM Computing Surveys**, New York, v. 41, n. 1, p. 1-25, 2008.

STÜTZLE, T.; HOOS, H. H. MAX–MIN ant system. **Future Generation Computer Systems**, Amsterdam, v. 16, n. 8, p. 889-914, 2000.

TALBI, E.-G. **Metaheuristics**: from design to implementation. Hoboken: John Wiley & Sons, 2009.

WHATWG. **HTML Living Standard**: server-sent events. [S. l.]: WHATWG, 2026. Disponível em: https://html.spec.whatwg.org/multipage/server-sent-events.html. Acesso em: [DATA DE ACESSO].

WOLPERT, D. H.; MACREADY, W. G. No free lunch theorems for optimization. **IEEE Transactions on Evolutionary Computation**, Piscataway, v. 1, n. 1, p. 67-82, 1997.

APÊNDICE A – ENDPOINTS DA API DA APLICAÇÃO ANTROUTE

Quadro 5 – Endpoints da API REST

Método	Rota	Finalidade
POST	/api/routes/optimize	Otimiza a rota com a ACO
POST	/api/routes/compare	Executa e compara os quatro algoritmos
POST	/api/routes/optimize-visual	Executa a ACO transmitindo as iterações via SSE
POST	/api/datasets/estimate	Estima o volume e a duração de uma tarefa
POST	/api/datasets/jobs	Cria uma tarefa de geração de dados
GET	/api/datasets/jobs[/{id}]	Lista as tarefas ou detalha uma tarefa
GET	/api/datasets/jobs/{id}/instances	Lista as instâncias de uma tarefa
GET	/api/datasets/jobs/{id}/configs	Resumo de desempenho por configuração
POST	/api/datasets/jobs/{id}/cancel | resume	Cancela ou retoma uma tarefa
DELETE	/api/datasets/jobs/{id}	Exclui uma tarefa e seus dados
GET	/api/datasets/jobs/{id}/export?level=...	Exporta execuções, instâncias, resumo ou ZIP

Fonte: elaborada pelo autor (2026).